

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

(ДГТУ)

КАФЕДРА «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ И АВТОМАТИЗИРОВАННЫХ СИСТЕМ»

ФУНКЦИИ. РЕКУРСИЯ

Практикум

по выполнению лабораторной работы №3

по дисциплине «Программирование на языке С++»

Ростов-на-Дону

2025

УДК 004.432

Составители: канд. техн. наук, доц. каф. А.А. Скляренко, ст. преп. Атаян А.М.

Практикум по выполнению практической работы по дисциплине «Программирование на языке С++». ДГТУ, г. Ростов-на-Дону, 2025 г.

В практикуме кратко изложены теоретические вопросы, необходимые для успешного выполнения практической работы, рабочее задание и контрольные вопросы для самопроверки.

Предназначено для обучающихся по направлению 02.03.03 «Математическое обеспечение и администрирование информационных систем», профиль «Большие данные и машинное обучение».

УДК 004.432

Печатается по решению редакционно-издательского совета
Донского государственного технического университета

Ответственный за выпуск:
зав. кафедрой «Программное обеспечение вычислительной техники и автоматизированных систем» Долгов В.В.

В печать __.__.20__ г.
Формат 60×84/16. Объем __ усл. п. л.
Тираж *50* экз. Заказ № (*указывает ИЦ ДГТУ)*

Издательский центр ДГТУ
Адрес университета и полиграфического предприятия:
344003, г. Ростов-на-Дону, пл. Гагарина, 1

© Донской государственный
технический университет, 2025

1 Теоретическая часть

1.1 Основные понятия. Функция

Функции, которые в других языках программирования иногда называются процедурами, подпрограммами, а в технологии объектно-ориентированного программирования – методами, позволяют выделить некоторую последовательность операторов в самостоятельный, функционально независимый именованный блок. Такой блок, будучи описан и протестирован на корректность единожды, может быть в последствие вызван (по своему имени) для выполнения отведенных ему функций многократно.

Функции позволяют программистам разбивать их большие (а иногда просто огромные) программы на маленькие независимые участки кода, что дает ряд неоспоримых преимуществ:

- независимые друг от друга функции могут разрабатываться различными программистами параллельно и независимо друг от друга;
- небольшие, выделенные в функции участки кода легче не только писать, но в дальнейшем понимать и дорабатывать;
- независимые друг от друга функции с небольшим числом аргументов значительно легче тестировать на корректность, что позволяет создавать более надежное программное обеспечение;
- готовые и прошедшие проверку на корректность независимые функции можно сочетать различным способом (как строительные блоки), используя их для создания самого разнообразного ПО.

Работа любой функции может зависеть от некоторых входных параметров, иногда называемых аргументами функции, и может передавать вызвавшему её коду некоторый (но один) результат вычислений, называемый возвращаемым значением или результатом (см. рисунок 1).

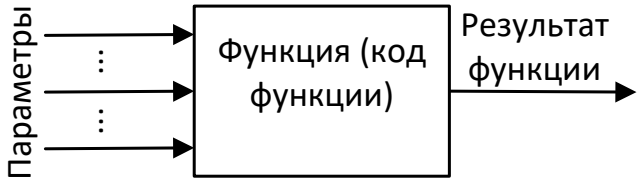


Рисунок 1 – Общая схема работы функции

1.2 Описание пользовательских функций

Любая функция должна быть обязательно объявлена перед своим первым использованием. В объявление функции включаются:

– тип возвращаемого ею результата, который в свою очередь, должен быть объявлен до объявления самой функции;

– её имя (должно быть правильным идентификатором языка C/C++);

– имена и типы данных всех её входных параметров.

Такое объявление называется прототипом и может размещаться как в основной программе, так и в отдельном файле заголовка (файлы с расширением «.h»). Прототип позволяет компилятору выполнять строгую проверку типов данных при последующих обращениях программы к функции.

Имя функции следует выбирать так, чтобы уже из него было приблизительно понятно, какие именно действия выполняются функцией. Часто в начало имени функции включают глагол, отражающий её основную (ключевую) задачу. Например, «getContentLength» (получить длину текстового содержимого), «readString» (считать строку), «compressUserData» (сжать пользовательские данные) и т.д. В разных коллективах программистов, кроме того, можно встретить различные правила (внутренние стандарты) или стили написания одного и того же имени функции.

На сегодняшний день наиболее часто используют три стиля оформления имен:

– c_case: Стиль, когда различные слова в имени функции начинаются с маленьких букв и разделяются символами подчеркивания («_»). Например: «get_content_length», «read_string».

– camelCase: Стиль при котором первое слово пишется с маленькой буквы, а все последующие за ним с заглавной. Например: «getContentLength», «readString».

– PascalCase: Стиль при котором все слова в имени пишутся с большой буквы. Например: «GetContentLength», «ReadString».

Соблюдение во всей программе выбранного стиля именования функций позволяет создавать единообразный и легкочитаемый код.

Операторы и выражения языка C/C++, описывающие действия, совершаемые функцией, записываются после ее объявления внутри фигурных скобок как составной оператор и называются телом функции. Все основные части описания простейшей функции, возвращающей значение своего целочисленного параметра, увеличенное на единицу, приведены на рисунке 2

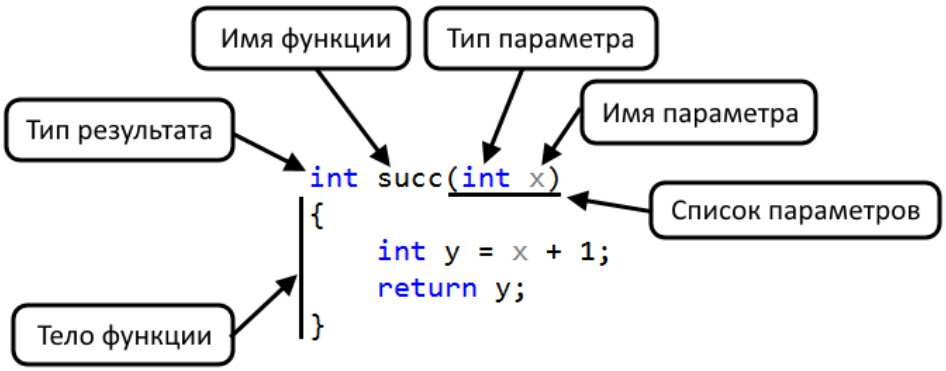


Рисунок 2 – Основные части описания функции

Таким образом, описание функции в общем случае должно соответствовать следующему шаблону:

```
<тип-результата> <имя-функции>(<список-параметров>)
{
    <тело-функции>
}
```

Этому же шаблону соответствует и уже встречаемая нами ранее в листингах функция «*main*». Эта особая функция должна быть в каждой программе, так как именно с нее начинается выполнение программы в целом.

В качестве примера рассмотрим описание и использование функции, принимающей на вход два целочисленных аргумента и возвращающей в качестве

результата их сумму (см. Листинг 1). Обратите внимание, что полное описание функции в коде расположено после точки её использования, но объявление функции находится до её использования. Такое разделение объявления и полного описания используется довольно часто (особенно в больших программам или библиотеках), однако оно не является обязательным и приведено в данном случае для примера.

Листинг 1 – Описание и использование функции сложения

```
#include "stdafx.h"
using namespace std;

//прототип функции сложения
int add(int x, int y);

int main(int argc, char* argv[])
{
     locale::global(locale( "Russian_Russia.1251" ));

     int a, b, c;
     cout << "Введите число a: "; cin >> a;
     cout << "Введите число b: "; cin >> b;

     c = add(a, b); //вызов функции сложения
     cout << a << " + " << b << " = " << c << endl;
     return 0;
}

//полное описание функции
int add(int x, int y)
{
     return x + y;
}
```

1.3 Параметры функций

Объявление любой функции обязательно включает в себя список ее параметров, который, впрочем, может быть пустым. Параметр можно рассматривать как внутреннюю переменную функции, принимающую в момент вызова функции значение соответствующего аргумента. В свою очередь аргумент – это некоторое значение (константное или вычисленное), передаваемое внутрь кода функции при ее вызове. Набор всех параметров, описанных при объявлении функции, называют списком параметров, а набор всех аргументов, передаваемых в функцию при ее вызове, – списком аргументов.

В профессиональной повседневной лексике программисты обычно не разделяют эти термины, понимая, о чем идет речь в зависимости от смыслового контекста. Одновременно с этим в русскоязычной литературе или в переводных изданиях можно также встретить синонимичные термины, когда параметр называют «формальным параметром», а аргумент – «фактическим параметром».

Когда список параметров функции пуст, то есть не содержит ни одного параметра, объявление функции все равно должно включать в себя круглые скобки, обрамляющие список параметров (несмотря на то, что его в данном случае нет). Допустимой, но несколько устаревшей формой записи пустого списка параметров является указание вместо него ключевого слова «*void*». Пример описания и вызова таких функций приведен в листинге 2.

Листинг 2 – Описание функций без параметров

```
void printHello(void)
{
     cout << "Привет, это функция без параметров" << endl;
}

int getTheNumber()
{
     return 42;
}
```

```
int main(int argc, char* argv[])
{
     locale::global(locale( "Russian_Russia.1251" ));

     printHello();
     cout << "... и число " << getTheNumber() << endl;

     return 0;
}
```

В случае, когда функция имеет более одного входного параметра, тип данных должен быть описан для каждого параметра отдельно. Будьте внимательны, это отличается от случая описания переменных, когда можно было указать несколько переменных одного типа через запятую (см. Листинг 3).

Листинг 3 – Описание функции с несколькими параметрами

```
//НЕВЕРНОЕ ОПИСАНИЕ
//Функция должна иметь два параметра, но для второго параметра
//с именем «y» не описан тип данных
int invalidSum(int x, y)
{
     return x + y;
}

//ВЕРНОЕ ОПИСАНИЕ
//Для обоих параметров указан тип данных
int validSum(int x, int y)
{
     return x + y;
}
```

При вызове любой функции для каждого переданного аргумента создается копия, значение которой присваивается соответствующему параметру функции. Таким образом, внутри функции мы оперируем не непосредственно аргументом (даже если он был представлен переменной), а его копией. Такой способ передачи

параметров называется «передачей по значению». При этом любые изменения параметра, сделанные внутри функции, меняют именно копию, не влияя на значение переменной-аргумента. Например, в коде из листинга 4 функция «*add*» изменяет внутри себя значение первого параметра «*x*», однако при выводе на экран значение переменной «*a*» остается прежним, хотя она и была передана в качестве аргумента, соответствующего параметру «*x*».

Листинг 4 – Передача параметров по значению

```
int add(int x, int y)
{
     x = x + y;
     return x;
}

int main(int argc, char* argv[])
{
     locale::global(locale("Russian_Russia.1251"));

     int a = 5, b = 7;
     int z = add(a, b);

     cout << a << "+" << b << "=" << z << endl;
     return 0;
}
```

Если посмотреть на функцию «*add*», то можно заметить, что для ее корректной работы не требуется изменять значение параметра «*x*», а сам код можно упростить, переписав его как

```
int add(int x, int y)
{
     return x + y;
}
```

Для подобных функций иногда бывает полезно явно указать компилятору, что значения параметров не должны меняться внутри функции. Для этих целей в языке C/C++ используется ключевое слово (модификатор) «*const*». При его применении компилятор проверит, что помеченный этим ключевым словом параметр не участвует ни в каких операциях (или вызовах функций), которые могут привести к изменению его значения. Функция «*add*» в этом случае примет вид

```
int add(const int x, const int y)
{
     return x + y;
}
```

Еще одной полезной возможностью является возможность указать для одного или нескольких последних параметров значений, которые они примут, если при вызове функции для них не будет указан передаваемый аргумент (т.е. проще говоря, аргумент будет опущен). В качестве примера, предположим, что для нашей функции «*add*» мы хотим позволить не указывать второй аргумент, возвращая при этом значение первого аргумента, увеличенного на единицу. Такое поведение эквивалентно сложению первого параметра с единицей. Функция и ее вызов в этом случае будут выглядеть как в нижеприведенном коде.

```
int add(const int x, const int y = 1)
{
     return x + y;
}
...
int a = 5, b = 7;
int z = add(a, b);  //z == 12
int y = add(a);          //y == 6, a == 5
```

Так как параметры по умолчанию могут располагаться только в конце списка параметров, то если у вас есть функция с, например, тремя параметрами, и вы решите указать значение по умолчанию для второго из них, вам придется также указать и значение по умолчанию для третьего (последнего) параметра. В противном случае вы получите ошибку при компиляции своей программы.

Несмотря на то, что передача параметров по значению применима в большинстве случаев, она обладает рядом ограничений. Во-первых, копирование значений при передаче аргументов позволяет возвращать значение вызывающей стороне только с использованием оператора «*return*». Во-вторых, при передаче больших структур данных в функцию постоянное копирование их значений будет приводить к накладным расходам не только по памяти, но и что более важно по скорости. Если функция вызывается часто задержки на копирования ее аргументов могут стать ощутимыми и их придется учитывать при оценки времени работы программы.

Рассмотрим, какие еще способы передачи параметров предлагают нам языки C/C++.

1.4. Передача параметров по ссылке

Если нам необходимо, чтобы при изменении значения параметра внутри функции одновременно происходило изменение и связанной с ним переменной-аргумента, можно объявить параметр как ссылочный. Для этого достаточно при объявлении функции поставить перед именем параметра символ «*&*».

Объявление параметра ссылочным приводит к тому, что на время вызова функции, такой параметр отождествляется с переменной, передаваемой в качестве аргумента (а не получает копию ее значения). Это обеспечивает синхронность изменения значения переменной-аргумента вслед за параметром. Обратной стороной медали при таком подходе становится невозможность передать в качестве аргумента константу или значение вычисленного выражения, так как в этом случае отсутствует именованная переменная, которую можно было бы отождествить с параметром.

В качестве первого примера использования ссылочного параметра, изменим поведение ранее рассматриваемой функции «*add*» и потребуем, чтобы сумма ее параметров не только возвращалась как результат работы функции, но и сохранялась в первом аргументе после ее завершения (см. Листинг 5).

Листинг 5 – Изменение значения аргумента функции через ссылку

```
int refAdd(int &x, const int y = 1)
{
     x += y;
     return x;
}
...
int a = 5, b = 7;
int z = refAdd(a, b);    //z == 12, a == 12
int y = refAdd(a);       //y == 13, a == 13
int x = refAdd(7, b);    //ОШИБКА: Ссылочный параметр не переменная
int w = refAdd(a+1, b);  //ОШИБКА: Ссылочный параметр не переменная
```

Общепринятой практикой является также использование ссылочных параметров для того, чтобы избежать ненужного копирования значительных объемов данных при вызове функций. Например, такие типы данных стандартной библиотеки языка C++ как «*string*» или «*vector*» при передаче в качестве аргументов создают свои копии. Учитывая, что они могут занимать достаточно большие объемы памяти, желательно передавать их по ссылке, когда надо, защищая переменные-аргументы от изменения модификатором «*const*». Например, как в коде ниже

```
void writeln(const string &line)
{
     cout << line << endl;
}
...
string s1 = "Длинная предлинная строка...";
writeln(s1);
```

Наконец последний общепринятый способ использования ссылочных параметров – передача через них дополнительных возвращаемых значений. Иногда это используется для обхода ограничения языка на возможность вернуть из функции только один результат. В приведенном ниже листинге 6 показан пример

описания и использования функции, одновременно вычисляющей значения синуса и косинуса для некоторого угла (в градусах).

Листинг 6 – Использования ссылочных параметров для возврата значений

```
void getSinCos(double degrees, double &sinOut, double &cosOut)
{
     static const double pi = 3.14159265358979323846;
     double radians = degrees * pi / 180.0;

     sinOut = sin(radians);
     cosOut = cos(radians);
}
...
double sin = 0.0;
double cos = 0.0;

getSinCos(30.0, sin, cos);

cout << "Sin = " << sin << '\n';
cout << "Cos = " << cos << '\n';
```

1.5. Передача параметров через указатель

Помимо передачи параметров по ссылке, передавать переменные-аргументы в функцию можно также с использованием указателей. Указатели подразумевают, что в функцию передается не значение самой переменной, а ее адрес в оперативной памяти. Использование указателя в качестве параметра мало чем отличается от использования ссылочных параметров. Параметры указатели, так же, как и ссылочные параметры, позволяют избегать копирования данных при передаче аргументов и дают возможность менять связанные с параметрами переменные-аргументы. Как правило, для этих целей в языке программирования C++ предпочтения отдают ссылочным параметрам как более безопасной и контролируемой альтернативе указателям. Однако параметры указатели имеют две особенности, которые могут быть очень полезны.

Во-первых, большое количество библиотечных функций в силу исторических причин принимают в качестве параметров указатели, а работа с динамической памятью почти полностью построена на них. Так что умение оперировать указателями при описании функций является очень важным.

Во-вторых, указатель в отличие от ссылки может быть пустым (равным NULL/nullptr), что может использоваться в алгоритмах как признак отсутствия обрабатываемого объекта. Поскольку язык C/C++ не позволяет ограничить передачу в качестве параметра пустого указателя, проверка такого типа параметров на пустоту становится, фактически, обязательной.

Предположим, что мы решили для обратной совместимости со старым кодом изменить объявление уже знакомой функции «*refAdd*» так, чтобы она принимала в качестве первого параметра указатель вместо ссылки. Тогда правильный вариант (с проверкой на пустоту указателя) будет выглядеть так, как показано в листинге 7.

Листинг 7 – Изменение значения аргумента функции через указатель

```
int refAdd(int *x, const int y = 1)
{
     int res = y;
     if (x != nullptr)
     {
          *x += y;
          res = *x;
     }

     return res;
}
...
int a = 5, b = 7;
int z = refAdd(&a, b);        //z == 12, a == 12
int y = refAdd(&a);           //y == 13, a == 13
int w = refAdd(nullptr, b);   //w == 7
```

Однако учтите, что, если в последнем примере вы захотите изменить не значение переменной, на которую указывает параметр-указатель, а сам указатель у вас ничего не получится, так как значение указателя передается «по значению», то есть копированием. Его изменение внутри функции не приведет к изменению адреса переменной за ее пределами. Фактически язык C/C++ всегда использует передачу параметров «по значению». Разница заключается в том, что при использовании указателей в параметрах, мы передаем дополнительную информацию (адрес переменной в памяти), позволяющую менять значение переменной за пределами функции, а при использовании ссылочных параметров компилятор делает то же самое вместо нас «за кулисами».

1.6. Массивы как параметры

Ну что ж, теперь мы можем передавать свои аргументы в функцию множеством различных способов. При рассмотрении всех этих способов негласно подразумевалось, что передаваемое значение представляет собой единый объект (возможно составной). Но что, если мы захотим передать в функцию массив, состоящий из множества однотипных объектов?

Существуют несколько способов передачи массивов, однако их всех объединяет одно и то же правило, – массивы всегда, явно или неявно, передаются в функцию по ссылке. Другими словами, каким бы из описанных ниже способов вы не воспользовались, ни в одном из них не происходит копирования значений элементов, и, следовательно, любые их изменения внутри функции будут так же изменять значения элементов массива-аргумента.

Что касается синтаксиса, то дела проще всего обстоят с массивами, размер которых известен на момент написания программы. Все, что в этом случае требуется, определить параметр функции так же, как это делалось при описании переменных. Например, в листинге 8 приведены функции, вычисляющие расстояние точки до начала координат и сложения двух векторов в трехмерном пространстве (три координаты, три элемента массива).

Листинг 8 – Описание параметра массива фиксированного размера

```
double length3(const double p[3])
{
     auto x = p[0], y = p[1], z = p[2];
     return sqrt(x*x + y*y + z*z);
}

void add3(double p1[3], const double p2[3])
{
     p1[0] += p2[0];
     p1[1] += p2[1];
     p1[2] += p2[2];
}

...

double p1[3] = { 1.0, 1.0, -1.0 };
double p2[3] = { -1.0, 1.0, -2.0 };
cout << "Длина вектора p1 = " << length3(p1) << endl;
cout << "Длина вектора p2 = " << length3(p2) << endl;
add3(p1, p2);
cout << "Длина вектора суммы = " << length3(p1) << endl;
```

В данном случае длина массива является частью определения типа данных, и передать в такие функции массивы с количеством элементов отличным от трех не получится (будет ошибка компиляции). Для проведения такой проверки количество элементов в массивах-аргументах так же должно быть известно в момент компиляции, что накладывает ограничения на применимость такого подхода и делает его не очень удобным с практической точки зрения.

Язык C/C++ позволяет при описании параметра функции не указывать размер массива внутри квадратных скобок. Такая форма записи параметра эквивалентна передачи указателя на первый элемент массива, в том числе по тем допустимым операциям, которые можно совершать с таким параметром внутри функции. Однако если мы передаем только указатель на начало массива, остается неясным,

сколько элементов содержит такой массив. Распространенной в этом случае практикой является передача длины массива в качестве дополнительного параметра как у функции «*printArray*» в листинге 9.

Листинг 9 – Передача массива через указатель с указанием длины

```
void printArray(const int *arr, int length)
{
     if( arr == nullptr || length <= 0 )
     {
          cout << "Массив пуст" << endl;
          return;
     }

     for (int i = 0; i < length; i++)
          cout << arr[i] << ' ';

     cout << endl;
}

...

int p1[3] = { 1, 1, -1 };
printArray(p1, 3);            //1 1 -1
printArray(NULL, 0);          //Массив пуст
```

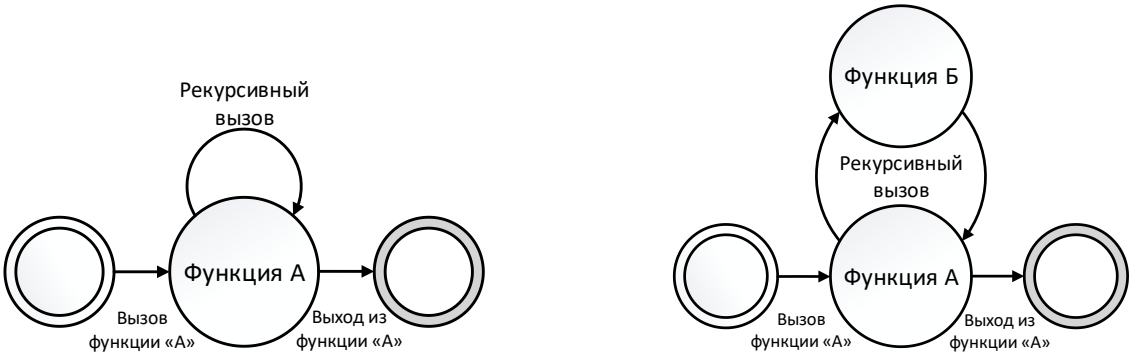
При использовании такого подхода следует соблюдать определенную осторожность, так как функция внутри себя не может проверить соответствие переданной длины реальному количеству элементов в массиве. Если переданное значение будет больше количества элементов в массиве, то в лучшем случае мы получим неверные результаты. Например, при вызове функции «*printArray*» следующим образом

```
int p1[3] = { 1, 1, -1 };
printArray(p1, 5);
```

на экран будут выданы два лишних числа, значения которых предсказать невозможно, так как они будут получены из области памяти, относящийся к другим переменным, а не к массиву «*p1*». Ситуация будет еще более плохой, если функция меняет элементы массива. В таком случае указание неверного значения длины приведет к уничтожению данных, не относящихся к обрабатываемому массиву и как следствие к некорректному и неопределенному поведению всей программы.

1.7. Организация рекурсивных вычислений

В программировании рекурсивной называется такая функция, решаемая задача которой сводится к одной или нескольким более простым, но аналогичным по форме задачам. В коде программы это выражается в виде вызова функцией самой себя напрямую (простая или прямая рекурсия, рисунок 3.1) или косвенно через некоторую другую функцию (сложная или косвенная рекурсия, рисунок 3.2).



1) простая или прямая рекурсия 2) косвенная рекурсия

Рисунок 3 – Виды рекурсий: 1 – простая; 2 – сложная или косвенная

Классическим примером применения простой рекурсии является функция вычисления факториала числа, выражающегося формулой $n! = \prod_{i=1}^{n} i = 1 \cdot 2 \cdot \ldots \cdot n$. Формулу легко переписать в виде $n! = n \cdot ((n-1) \cdot \ldots \cdot 2 \cdot 1)$, что в свою очередь равно $n! = n \cdot (n-1)!$. Из последней формулы хорошо видно, что факториал любого заранее заданного положительного числа выражается через факториал числа, меньшего заданного на единицу, то есть через ту самую «простую, но аналогичную по форме задачу». Код функции по такой формуле получается очень простым, понятным и пишется, что называется, «в лоб» (Листинг 10).

Листинг 10 – Рекурсивное вычисление факториала

```
unsigned long factorial(unsigned n)
{
   if( n > 1 )
     return n * factorial(n-1);
   else
     return 1;
}
```

На примере функции вычисления факториала также очень хорошо видны два правила, которым должна удовлетворять «хорошая» рекурсивная функция:

– рекурсия всегда должна осуществляться по какому-либо условию, которое в один из моментов не должно сработать, приведя к завершению рекурсии (для функции факториала этим условием является требование «*n > 1*»);

– рекурсивный вызов всегда должен происходить с параметрами, соответствующими более простой решаемой задаче, параметрами, ведущими к завершению рекурсии (в случае факториала это требование выполняется вычитанием единицы из значения переменной «*n*»).

Пренебрежение двумя приведенными выше правилами часто приводит к бесконечной рекурсии. Языки C/C++ для любого вызова функции всегда тратят немного памяти внутри специальной области, называемой стеком, чтобы разместить там копии передаваемых функции аргументов. Появление бесконечной рекурсии всегда означает постепенное исчерпание этой памяти и следующее за этим ошибочное завершение программы. Следите за тем, чтобы указанные правила всегда выполнялись!

Простая одиночная рекурсия, по примеру приведенной выше функции вычисления факториала, всегда является взаимозаменяемой с итерационными циклами, которые не имеют проблем с расходом памяти стека, даже в случае бесконечного зацикливания. Например, ту же самую функцию вычисления факториала легко переписать с использованием цикла «*for*», как это показано в листинге 11.

Листинг 11 – Цикличный вариант функции вычисления факториала

```
unsigned long factorial2(unsigned n)
{
     unsigned long res = 1;
     for (unsigned i=2; i <= n; i++) res *= i;
     return res;
}
```

Однако очень часто, рекурсивные алгоритмы даже в таких простых случаях оказываются более простыми для понимания. В случае же множественных (когда функциях вызывается сама себя несколько раз) или косвенных рекурсий их преобразование делает код программы настолько запутанным, что такое преобразование часто теряет всякий смысл, а иногда приводит к перерасходу памяти. Самым простым примером рекурсивного алгоритма, преобразование которого в цикл невозможно, является алгоритм обхода классического дерева, построенного без дополнительной информации в узлах.

В качестве дополнительных примеров рекурсивных алгоритмов рассмотрим часто встречающуюся задачу нахождения наибольшего общего делителя двух чисел и алгоритм быстрого (двоичного) поиска элемента в отсортированном массиве.

Алгоритм решения задачи о наибольшем общем делителе (НОД, GCD) двух чисел описан еще Евклидом. Он задается рекурсивной формулой

$$\gcd(x, y) = \begin{cases} x, \textit{если } y = 0 \\ \gcd(y, \textit{remainder}(x, y)), \textit{если } y > 0 \end{cases},$$

где функция *remainder* – это остаток от деления *x* на *y* (то есть операция «%» в языке C/C++).

Так же, как и в случае факториала, формула довольно просто выражается в коде на C++ (см. Листинг 12).

Листинг 12 – Рекурсивный поиск наибольшего общего делителя

```
unsigned gcd(unsigned x, unsigned y)
{
     return y > 0 ? gcd(y, x % y) : x;
```

```
}
```

Алгоритм двоичного поиска также является одним наиболее важных алгоритмов в программировании, так как позволяет очень быстро (более строго, за логарифмическое время) найти заданный элемент в заранее отсортированном массиве или убедиться, что такого элемента там нет. Хотя основное ядро алгоритма может быть реализовано в виде условного цикла, в данном пособии мы рассмотрим его рекурсивную реализацию. Как уже было сказано раньше, важным шагом при описании рекурсии является способ разбиения задачи на более простые подзадачи и определение граничного условия, при достижении которого рекурсия будет прекращена.

Итак, пусть нам дан некоторый отсортированный (упорядоченный) массив произвольной длины, в котором мы хотим найти заранее заданный элемент (см. Рисунок 4).

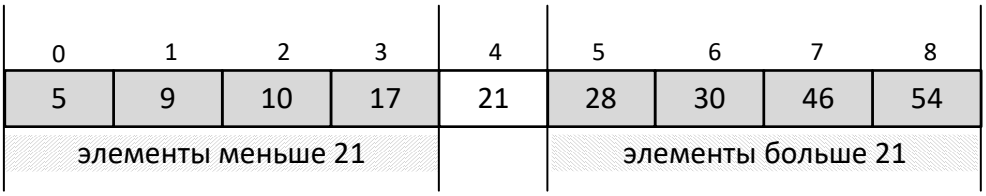


Рисунок 4 – Разбиение задачи двоичного поиска на две подзадачи

Первым делом возьмем в массиве центральный элемент (на рисунке 4 это число 21) и сравним его со значением, которое нам необходимо найти. При сравнении у нас могут возникнуть три ситуации:

Центральный элемент массива равен искомому. Это означает, что требуемый элемент найден и результатом работы алгоритма становится номер этого элемента в массиве (на рисунке 4 это 4-я ячейка). Данный случай является одним из граничных условий, завершающих рекурсию.

Если центральный элемент массива больше искомого, то требуемый нами элемент если и присутствует в массиве, то только слева (в части массива с меньшими индексами) от центрального. Такой вывод естественным образом следует из предположения, что массив исходно отсортирован, а значит все

элементы справа от центрального либо равны ему, либо больше него, а значит они также и больше искомого.

Если же центральный элемент меньше искомого, то по точно такой же логике что и в предыдущем случае поиск можно продолжать только в правой части массива (в части с номерами элементов большими, чем у центрального).

Заметим, что и левая, и правая по отношению к центральному элементу части массива сами по себе являются отсортированными массивами с меньшим чем у исходного числом элементов (см. Рисунок 4). Это позволяет нам применить в выбранной части массива (только левой или только правой) тот же самый алгоритм поиска с выбором центрального элемента, повторяя деление до тех пор, пока мы не найдем требуемое значение.

Если же искомого значения в исходном массиве нет, то каждый раз уменьшая область поиска, мы дойдем до ситуации, когда вновь выбранная часть массива не будет содержать элементов. Этот случай будет вторым граничным условием для завершения рекурсии. В качестве результата в этом случае в нашем примере мы будем возвращать -1 как заведомо неверный индекс в массиве.

Код программы для поиска по описанному алгоритму приведен ниже в листинге 13.

Листинг 13 – Быстрый (двоичный) поиск элемента в массиве

```
int _binary_search(int *data, int fromIdx, int toIdx, int element)
{
     int mid = fromIdx + (toIdx - fromIdx) / 2;

     if (fromIdx > toIdx) return -1;
     else if (data[mid] == element) return mid;
     else if (data[mid] > element)
          return _binary_search(data,fromIdx,mid-1,element);
     else
          return _binary_search(data,mid+1,toIdx,element);
}

int binary_search(int *data, int length, int element)
```

```
{
    return _binary_search(data, 0, length - 1, element);
}
```

2 Цель работы

Изучение концепции функций и рекурсии в языке программирования C++. Формирование навыков создания и использования функций, а также понимание принципов рекурсивного подхода при решении задач. Освоение синтаксиса определения и вызова функций, а также развитие навыков разбиения сложных задач на подзадачи с использованием рекурсии.

3 Задание к лабораторной работе

Разработать программу, реализовав в виде функции(й) один из вариантов задания в соответствии с таблицей 1. Необходимые для работы программы параметры, вводить с клавиатуры. Результаты вычислений выводить на экран в стандартный поток вывода (консоль). Второе задание выполняется на основе рекурсивной функции.

Проект должен состоять как минимум из трех файлов: lab.h, lab.cpp, main.cpp.

Таблица 1 – Задание в лабораторной работе

№ вар-та	Задание
1	1. Пусть даны координаты вершин треугольника (в виде массивов координат точек «xt» и «yt») и координаты некоторой точки внутри него. Найти расстояние от данной точки до ближайшей стороны треугольника. 2. Вычислить произведение элементов одномерного массива.
2	1. Пусть на вход функции подаются координаты вершин двух треугольников A и B в виде массивов xa, xb, ya и yb. Определить, какой из треугольников имеет большую площадь, вернув 1, если большую площадь имеет треугольник A, и 2, если B. 2. Вычислить значение полинома n по формуле $P_n = \sum_{i=0}^{n} a_i x^i = a_0 + x\,(a_1 + x(a_2 + \cdots + xa_n)$

№ вар-та	Задание
3	1. Описать функцию stepen(x, n) от вещественного x и целого n, вычисляющую (посредством умножения) величину x^n, и использовать ее при создании функции F(k, a) возвращающей значение $F(k,a) = 2.7^k + (a+1)^{-5}$. 2. Найти максимальный элемент в массиве $a_1, \dots, a_n$, используя метод деления пополам max(max ($a_1, \dots, a_{\frac{n}{2}}$) , max ($a_{\frac{n}{2}+1}, \dots, a_n$))
4	1. Даны отрезки с длинами a, b, c и d. Для каждой тройки отрезков, из которых можно построить треугольник, напечатать площадь данного треугольника. Определить функцию Triangle(double x, double y, double z), возвращающую площадь треугольника со сторонами x, y и z, если такой треугольник существует, и -1, если он невозможен. 2. Вычислить сумму $S_n = \sum_{i=1}^{n} \frac{i^3}{i+1}$
5	1. Длины сторон некоторого треугольника заданны параметрами a, b и c. Найти медианы треугольника, сторонами которого являются медианы исходного треугольника. Длина медианы, проведенной к стороне a, равна $\frac{1}{2}\sqrt{2b^2+2c^2-a}$. 2. Описать рекурсивную функцию NOD(A, B) целого типа, находящую наибольший общий делитель (НОД) двух целых положительных чисел A и B, используя алгоритм Евклида: НОД(A, B) = НОД(B, A mod B), если B = 0; НОД(A, 0) = A. С помощью этой функции найти НОД(A, B), НОД(A, C), НОД(A, D), если даны числа A, B, C, D.
6	1. Два натуральных числа называются «дружественными», если каждое из них равно сумме всех делителей другого, за исключением его самого (таковы, например, числа 220 и 284). Напечатать все пары «дружественных» чисел, не превосходящих заданного натурального числа. В готовой программе должна быть как минимум одна функция (может быть больше при необходимости), принимающая на вход два числа и возвращающая целое ненулевое значение, если исходные числа являются «дружественными». 2. Вычислить сумму $S_n = \sum_{i=1}^{n} \frac{2i+1}{i^2}$
7	1. Дано четное число n > 2. Проверить для этого числа гипотезу Гольдбаха. Эта гипотеза заключается в том, что каждое четное n, большее двух, представляется в виде суммы двух простых чисел. Воспользоваться функцией распознавания простых чисел. 2. Дана последовательность натуральных чисел (одно число в строке), завершающаяся числом 0. Выведите все нечетные числа из этой последовательности, сохраняя их порядок. В этой задаче нельзя использовать глобальные переменные и передавать какие-либо параметры в рекурсивную функцию. Функция получает данные, считывая их с клавиатуры. Функция не возвращает значение, а сразу же выводит результат на экран. Основная программа должна состоять только из вызова этой функции.

№ вар-та	Задание
8	1. Даны целые числа m, n. Вычислить $c = \frac{m!}{n!(m-n)!}$. Для вычисления факториала использовать отдельную функцию. 2. Вычислить значение $x = \sqrt{a}$, используя формулу $x_n = \frac{1}{2}(x_{n-1} + \frac{a}{x_{n-1}})$, в качестве начального приближения использовать значение $x_0 = \frac{1+a}{2}$
9	1. Заданы два одномерных массива A и B размером N. Найти $S = \left\{ \begin{array}{l} \prod_{i=1}^{N} a_i, если \prod_{i=1}^{N} b_i > 0 \\ \prod_{i=1}^{N} (a_i + b_i), если \prod_{i=1}^{N} b_i \le 0 \end{array} \right\}$. Для вычисления произведений использовать функцию. 2. Описать рекурсивную функцию Combin2(N, K) целого типа, находящую C(N, K) — число сочетаний из N элементов по K — с помощью рекуррентного соотношения: C(N, 0) = C(N, N) = 1, C(N, K) = C(N − 1, K) + C(N − 1, K − 1) при 0 < K < N. Параметры функции — целые числа; N >0, 0 ≤ K ≤ N. Считать, что параметр N не превосходит 20. Для уменьшения количества рекурсивных вызовов по описать вспомогательный двумерный массив для хранения уже вычисленных чисел C(N, K) и обращаться к нему при выполнении функции Combin2. С помощью функции Combin2 найти числа C(N, K) для данного значения N и пяти различных значений K.
10	1. Даны действительные s, t, A(12). Вычислить $z = P(1) - P(t) + P^2(s-t) - P^3(t)$, используя в качестве подпрограммы функцию $P(x) = a_{12}x^{12} + a_{11}x^{11} + ... + a_1$. 2. Вычислить сумму $S_n = \sum_{i=1}^{n} \frac{\sin i}{2i-1}$
11	1. Вычислить $z = \frac{e^{\lvert x_{\max}\rvert} - e^{\lvert y_{\max}\rvert}}{\sqrt{\lvert x_{\max} y_{\max}\rvert}}$, где $x_{\max}$ – наибольший элемент массива X(20); $y_{\max}$ – наибольший элемент массива Y(15). Для вычисления наибольшего элемента массива использовать функцию. 2. Описать рекурсивную функцию PowerN(X, N) вещественного типа, находящую значение N -й степени числа X по формулам: $X^0 = 1$, $X^N = (X^{N/2})^2$ при *четных* $N > 0$, $X^N = X*X^{N-1}$ при *нечетных* $N > 0$, $X^N = 1/X^{-N}$ при $N < 0$. ($X = 0$ — вещественное число, N — целое; в формуле для четных N должна использоваться операция *целочисленного деления*). С помощью этой функции найти значения X^N для данного X при пяти данных значениях N.
12	1. Составить программу для вычисления функции $U = e^{x_1+y_1} + e^{x_2-y_2}$ где x_1, y_1 – корни уравнения $ax^2 + bx - 1.5 = 0$; y_1, y_2 – корни уравнения $2y^2 - y + c = 0$. Корни находить в функции. Если корни мнимые, то считать их равными нулю. 2. Вычислить сумму $S_n = \sum_{i=1}^{n} \frac{\cos i}{4i-1}$

№ вар-та	Задание
13	1. Даны действительные числа a, b. Вычислить выражение $z = F(\sin a, b) + F(\cos b, a) + F(\sin^2 a, a-1) + F(\sin^2 a, \cos b)$, используя в качестве подпрограммы функцию $F(u,t) = \begin{cases} u + \sin t, если\, u > 0 \\ u + t, если\, u \le 0 \end{cases}$ 2. Описать рекурсивную функцию RootK(X, K, N) вещественного типа, находящую приближенное значение корня K-й степени из числа X по формуле: $Y_0 = 1$, $Y_{N+1} = Y_N - (Y_N - X/(Y_N)^{K-1})/K$, где Y_N обозначает RootK(X, K, N) при фиксированных X и K. Параметры функции: X (> 0) — вещественное число, K (> 1) и N (> 0) — целые. С помощью функции RootK найти для данного числа X приближенные значения его корня K-й степени при шести данных значениях N.
14	1. Даны массивы действительных чисел A(20), B(15), C(10). Вычислить $M = \frac{S+T+K}{2}$, где S, T, K – суммы положительных элементов массивов A, B, C соответственно. Для вычисления суммы положительных элементов использовать функцию. 2. Вычислить значение функции $F(x) = \sum_{i=1}^{n} \frac{sin^2(ix)}{i^3}$
15	1. Даны вещественные матрицы A, B, C, D размером MxN. Составить процедуру сложения двух матриц и использовать ее в программе для нахождения $P = (A+B) - (C+D)$. 2. Вычислить среднее значение элементов одномерного массива

4 Материально-техническое обеспечение работы

Аудитория для проведения лабораторных занятий должна быть укомплектована специализированной мебелью и индивидуальными компьютерами следующей минимальной комплектации:

– Процессор: не менее двух исполнительных ядер, совместимый с системой команд x86 и x64, с поддержкой аппаратной виртуализации.

– Оперативная память: не менее 8 Гб.

– Монитор: не менее 24" (дюймов) по диагонали.

– Наличие локальной сети со скоростью обмена не менее 1 Гб/сек.

– Наличие доступа в сеть Интернет со скоростью не менее 1 Мбит/сек.

– Наличие клавиатуры и манипулятора «мышь».

На компьютерах должно быть установлено следующее программное обеспечение:

– Операционная система: Microsoft Windows 10 (или выше) и/или одна из операционных систем семейства Linux, допускающих установку среды исполнения для C++. Допускается конфигурация, когда одна из операционных систем установлена внутри виртуальной машины (гипервизора).

– Среды разработки и исполнения программ: Microsoft Visual Studio 2022 Community Edition (или выше), Visual Studio Code, Dev-C++.

5 Порядок выполнения и сдачи работы

Для выполнения лабораторной работы рекомендуется придерживаться следующего порядка выполнения:

1. Ознакомится с темой и целями лабораторной работы
2. Изучить теоретический материал
3. Подготовить рабочее окружение
4. Разработать программный код лабораторной работы в соответствие с заданием
5. Произвести проверку работоспособности, выполнить тестирование и отладку кода
6. Проанализировать полученные результаты и примененные в ходе решения подходы
7. Выполнить самооценку и рефлексию
8. Сдать лабораторную работу преподавателю и получить от него обратную связь

При сдаче студентом лабораторной работы основным отчетом выступает исходный код самостоятельно созданной в процессе выполнения работы программы.

Исходный код должен быть отформатирован согласно принятым для используемого языка программирования стандартам. Является желательным наличие в исходном коде комментариев, описывающих основные части программы и особенности их функционирования. В то же время студент должен быть готов

объяснить работу программы в целом и каждую отдельную ее часть при полном отсутствии комментариев (например, они могут быть удалены преподавателем при сдаче работы).

Обязательным условием сдачи является умение студента самостоятельно восстановить любой участок исходного кода программы (но не более 20 строк подряд) после его удаления. Удаленный участок должен быть самостоятельно набран студентом заново в присутствие преподавателя. При восстановлении участка запрещается использовать операции Undo (Отменить) текстовых редакторов или переписывание кода участка из других источников.

Исходный код сдаваемой программы должен быть представлен в электронном виде.

Сдача исходного кода ранее сдававшихся программ или программ, код которых выложен в сети Интернет, не допускается.

6 Контрольные вопросы к лабораторной работе

1. С помощью какой функции можно вернуть результат вычислений из функции?
2. Чем отличается передача параметров по значению от передачи по ссылке?
3. Какие библиотеки необходимо подключить для работы с функциями ввода/вывода (например, cout и cin)?
4. В чем разница между параметром и аргументом функции?
5. Как объявить параметр по умолчанию и почему он должен быть в конце списка?
6. С какой функции начинается выполнение программы на C++?
7. Что делает ключевое слово const при объявлении параметра?
8. Какие ошибки могут возникнуть при передаче массива в функцию без указания его длины?
9. Почему изменения элементов массива внутри функции влияют на исходный массив?

10. Какие правила должны соблюдаться для корректной работы рекурсивной функции?

11. Какие стили именования функций существуют? Приведите примеры.

12. Что произойдет, если передать в функцию refAdd нулевой указатель?

7 Перечень использованных информационных ресурсов

1. Подбельский, В. В. Стандартный Си++ : учебное пособие / В. В. Подбельский. — Москва : Финансы и статистика, 2022. — 688 с. — ISBN 978-5-00184-081-7. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/book/309404 (дата обращения: 06.05.2024). — Режим доступа: для авториз. пользователей.

2. Рацеев, С. М. Программирование на языке Си / С. М. Рацеев. — 2-е изд., стер. — Санкт-Петербург : Лань, 2023. — 332 с. — ISBN 978-5-507-47236-9. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/book/351863 (дата обращения: 06.05.2024). — Режим доступа: для авториз. пользователей.